TRABALHO DE CONCLUSÃO DE CURSO II · CIÊNCIA DA COMPUTAÇÃO

Mônadas e Efeitos Algébricos em Linguagens Funcionais

Uma análise comparativa em legibilidade, composabilidade, poder de abstração e desempenho

Mônadas · Haskell Efeitos algébricos · OCaml 5

Diego Marchioni
Orientador: Marco Rodrigo

Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Exatas e Informática · dezembro de 2026

CONTEXTO

Dois mecanismos para efeitos

HASKELL · MTL

Mônadas e transformadores

`exceção · estado · IO`

Cada efeito é uma camada da pilha e aparece no tipo da função.

OCAML 5 · NATIVO DESDE 2022

Efeitos algébricos e handlers

```
perform (Get x)
```

O código pede a operação, e o handler decide o que ela faz.

Na teoria, os dois expressam as mesmas coisas para estes efeitos (Schrijvers et al., 2019). Falta medir quanto custa escrever e executar com cada um.

MÉTODO

Cada mecanismo contra uma base

Handlers (OCaml 5)	÷	Imperativa (OCaml 5)	=	**razão do OCaml**
Mônadas (Haskell)	÷	IORef (Haskell)	=	**razão do Haskell**

Comparar tempos entre linguagens mediria também os compiladores. Por isso, cada mecanismo é comparado com uma base simples na própria linguagem, com a mesma estrutura de dados.

$$razão = 1 + \Delta / base$$

Δ é o custo a mais do mecanismo. Uma base muito barata deixa a razão grande; por isso a comparação entre linguagens é só indicativa.

MÉTODO

Questões de pesquisa

QP1

Legibilidade

Linhas, operações de efeito e aninhamento de *eval*, como medidas indiretas.

QP2

Composabilidade

Partes do código existente que mudam ao acrescentar um quarto efeito.

QP3

Poder de abstração

Código de infraestrutura fora de *eval* e quanto dele é reaproveitado.

QP4

Desempenho

Tempo e memória contra a base da mesma linguagem, em quatro cenários.

Comparações de código só dentro da mesma linguagem: handlers × imperativa, mônadas × IORef.

CASO DE USO

Um interpretador com três efeitos

OITO CONSTRUTORES

`Num` `Var` `Add` `Mul` `Div`

`Let` `Seq` `Print`

Estado

Var lê uma variável e Let guarda um valor.

Exceção

Divisão por zero e variável inexistente dão erro.

Saída

Print mostra um valor na saída.

Especificação formal escrita antes do código. Duas versões são equivalentes se dão o mesmo valor, o mesmo estado, a mesma saída e o mesmo erro.

Simplificações: variáveis globais, o erro sempre encerra a expressão, um único programa.

IMPLEMENTAÇÃO

Quatro versões e a verificação

Linguagem	Mecanismo	Linha de base
OCaml 5	Handlers para Get, Set, Fail e Output	Imperativa: tabela hash e exceções
Haskell	Mônadas: pilha de exceção, estado e IO	Variável mutável (IORef), mesma estrutura de dados

Nada é medido antes de os testes passarem nas quatro versões.

61/61
testes passando em cada linguagem

8/8
erros colocados de propósito, todos detectados

12
cargas de expressões idênticas nas duas linguagens

IMPLEMENTAÇÃO

O mesmo trecho de eval, com e sem handlers

HANDLERS · INTERP_EFFECTS.ML

```
| Var x → (
    match perform (Get x) with
    | Some v → v
    | None → perform (Fail (msg_unbound x)))
```

IMPERATIVA · INTERP_IMP.ML

```
| Var x → (
    match Hashtbl.find_opt env x with
    | Some v → v
    | None → raise (Runtime_error (msg_unbound x)))
```

Cada operação direta da base deu lugar a exatamente uma operação de efeito.

Código completo, testes e scripts: github.com/DiegoMarchioni/TCC, pasta codigo/

RESULTADOS · QP1

Legibilidade de eval

Versão	Linhas de eval	Operações de efeito	Aninhamento
Handlers (OCaml)	29	5	2
Imperativa (OCaml)	29	5	2
Mônadas (Haskell)	27	18	1
IORef (Haskell)	27	18	1

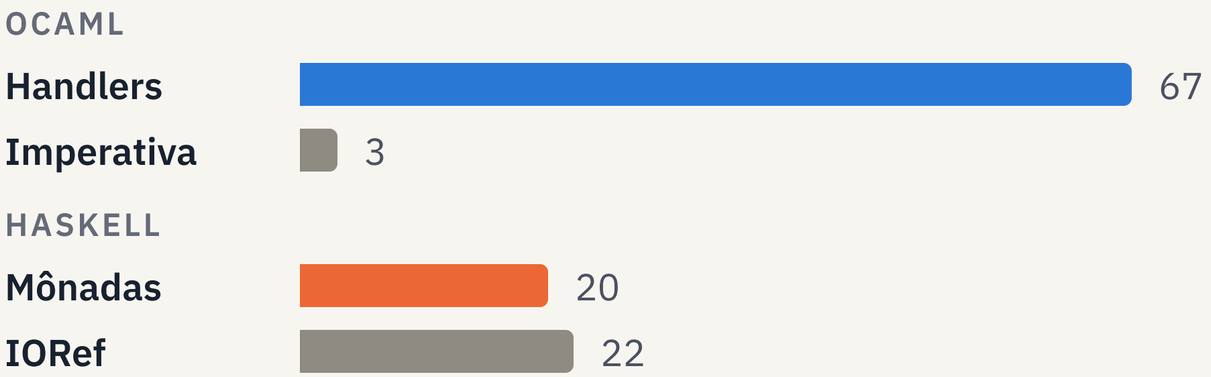
Os números coincidem também construtor a construtor. Pelas três medidas, o mecanismo não muda a leitura de eval.

Contagem automática sobre o código formatado. Cada linguagem tem as suas operações de efeito, por isso só se compara dentro da mesma.

RESULTADOS · QP2 E QP3

Composabilidade e infraestrutura

Linhas de infraestrutura fora de eval



18 das 67 linhas dos handlers servem só aos testes.

Reaproveitamento só nos handlers: os de estado e de exceção são usados em dois lugares.

Pontos de modificação para o quarto efeito

2 Handlers	0 Imperativa
2 Mônadas	0 IORef

Só os mecanismos mudaram declarações existentes; as bases só ganharam funções novas.

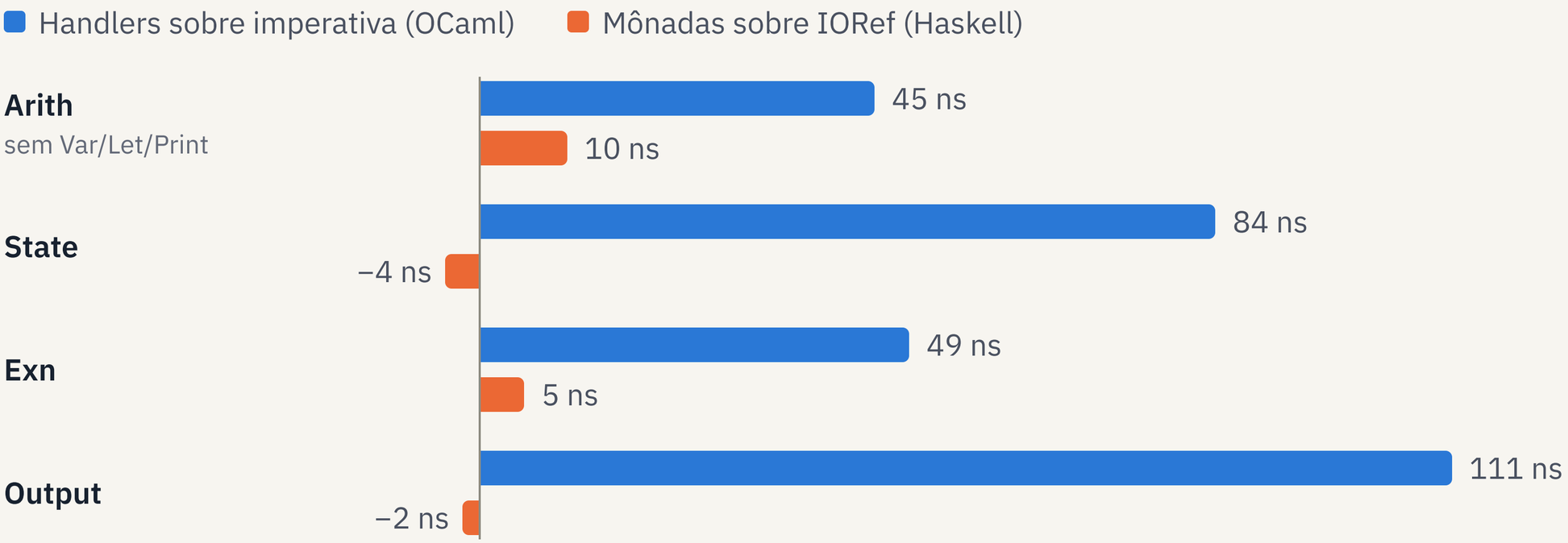
QP4 · OCAML, CENÁRIO ARITH

45 ns

por expressão, só para instalar três handlers, sem nenhuma operação de efeito.

No Haskell, a maior diferença sobre a base foi de 10 ns por expressão.

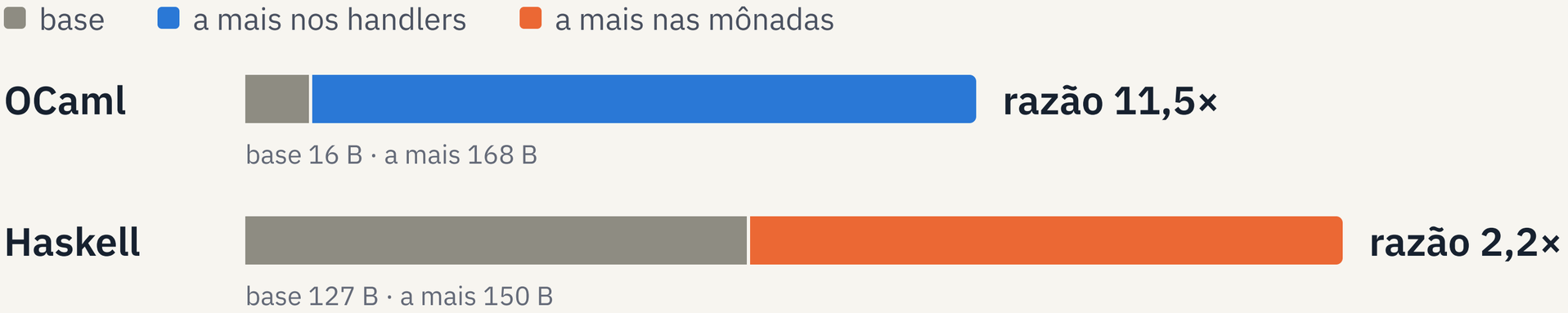
Custo de tempo por expressão



Tempo a mais que a base da mesma linguagem, em ns por expressão (mediana de 3 conjuntos de expressões). Razões: OCaml de 1,59 a 4,26; Haskell de 0,97 a 1,08.

RESULTADOS · QP4

Memória por expressão



A razão do OCaml é alta porque a base quase não usa memória: em bytes, o que cada mecanismo usa a mais é parecido.

Cenário Arith, bytes alocados por expressão.

RESULTADOS

Achados não previstos

OCAML 5.2 · EXCEÇÃO

Abandonar a computação interrompida vaza memória

1,2 GB
abandonando a computação

6 MB
com discontinue

Memória usada após 2 milhões de erros. O meu código usa discontinue.

MEDIÇÃO

Passar no critério de qualidade não garante estabilidade

até 28%
de deslocamento nas razões

Com um tempo de coleta 4 vezes menor, todas as medições passaram no critério ($R^2 \geq 0{,}95$), e mesmo assim as razões mudaram.

DISCUSSÃO

Frente à literatura e ao TCC I

Fonte	O que a fonte mede	Neste trabalho
Sivaramakrishnan et al., 2021	cerca de 23 ns para instalar um handler	45 ns para instalar três handlers
Kammar et al., 2013; Xie e Leijen, 2020	mônadas de 1,0 a 1,3× uma versão sem efeitos	0,97 a 1,08× a base
TCC I (previsão)	handlers abaixo de 1,5×; mônadas acima de 2×	aconteceu o contrário nos dois casos

Cada trabalho mede uma situação diferente; a comparação é só de ordem de grandeza.

CONCLUSÃO

Conclusões e trabalhos futuros

Escrita

O custo está no código em volta de eval: nos handlers, onde eles são instalados; nas mônadas, no tipo da pilha, que muda a cada efeito.

Execução

Handlers custam ao ser instalados e a cada operação, mesmo sem efeito. As mônadas, otimizadas pelo compilador, quase não custam.

Limites

Um único programa, uma máquina virtual e versões específicas das linguagens.

TRABALHOS FUTUROS

Comparar os dois numa mesma linguagem · outros programas e efeitos · legibilidade medida com pessoas

Obrigado.

Perguntas e comentários da banca

`github.com/DiegoMarchioni/TCC`

Código, resultados e artigo; make tudo refaz cada número do texto.

Diego Marchioni · Orientador: Marco Rodrigo · PUC Minas

APÊNDICE

Tabela 2 completa

Tempo, em ns por expressão

	OCaml			Haskell		
Cenário	base	Δ	razão	base	Δ	razão
Arith	77	45	1,59	96	10	1,08
State	87	84	1,98	346	−4	0,99
Exn	77	49	1,63	95	5	1,05
Output	34	111	4,26	41	−2	0,97

Alocação, em bytes por expressão

	OCaml			Haskell		
Cenário	base	Δ	razão	base	Δ	razão
Arith	16	168	11,5	127	150	2,2
State	48	382	9,0	117	9	1,1
Exn	16	187	12,7	119	136	2,1
Output	64	360	6,6	104	8	1,1

IC de 95% da razão de tempo: nenhum inclui 1 no OCaml; no Haskell, só o de Arith o exclui ([1,04; 1,18]). Dispersão entre sementes de até 18,3% no tempo e inferior a 0,1% na alocação. Menor R^2: 0,954 no Bechamel e 0,973 no Criterion. Medianas de 3 sementes.

APÊNDICE

Protocolo de medição

Item	Configuração
Máquina	Intel Xeon @ 2,10 GHz, 4 vCPU, máquina virtual de nuvem, Ubuntu 24.04.5; afinidade fixa num núcleo
Compilação	OCaml 5.2.1 + flambda com -O3; GHC 9.4.7 com -O2; flags iguais para mecanismo e base
Harnesses	Bechamel 0.5.0, cota de 20 s e limit 5000; Criterion 1.6.1.0, 1.000 reamostragens
Cargas	4 cenários × 10.000 expressões × 3 sementes (42, 43, 44), geradas fora da região medida
Instalação	três handlers por expressão no OCaml; a pilha executada uma vez por expressão no Haskell
Regras	oráculo como pré-condição; mesmo resultado conferido no harness; ordem alternada; $R^2 \geq 0{,}95$
Repetição	um par com R^2 0,949 foi remedido (0,962), com o descarte registrado
Execução	pela ferramenta de IA, em sessão na nuvem, como declara o artigo
Reprodução	make tudo: testes, métricas, campanha, sensibilidade, artigo e figuras

APÊNDICE

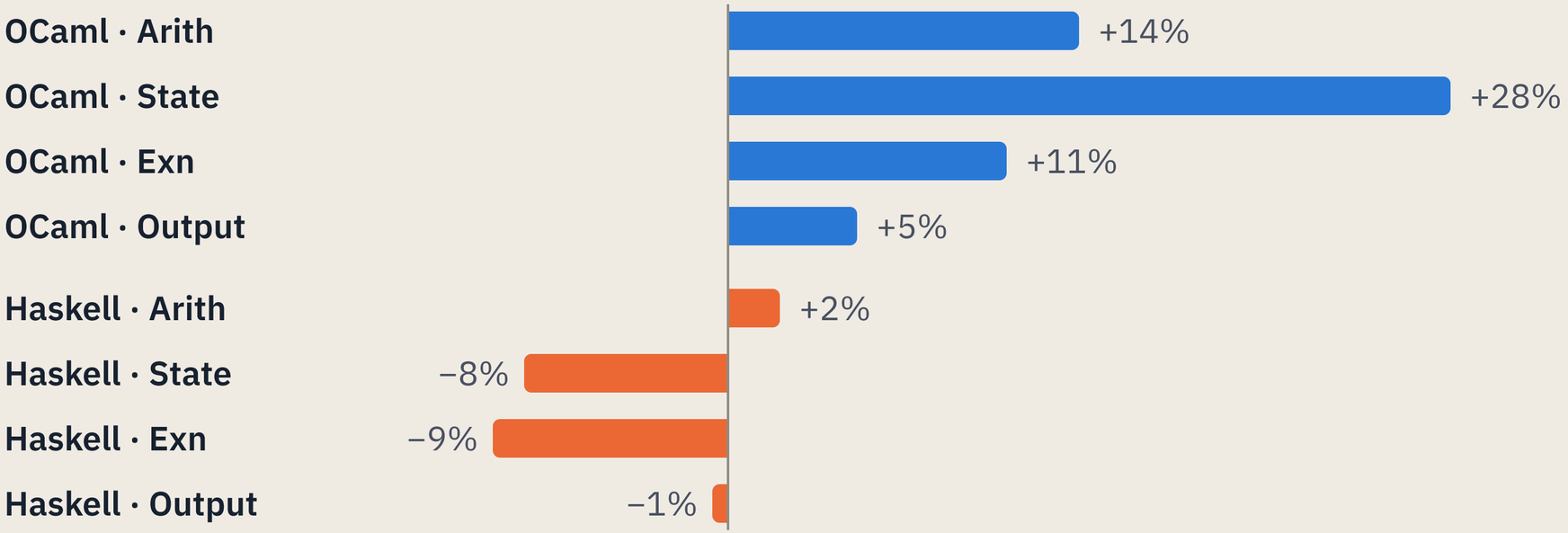
O quarto efeito em cada versão

Versão	O que mudou em declarações existentes	Pontos	Adições
Handlers	run e run_acc passam a instalar o handler de Input	2	2
Imperativa	nada; a leitura é uma função nova	0	1
Mônadas	o tipo da pilha ganha StateT [Int] e run passa a executá-la; a leitura usa dois lift	2	1
IORef	nada; a leitura é uma função nova	0	1

Ponto de modificação: declaração preexistente cujo texto mudou. Excluídos por serem iguais nas quatro versões: o construtor Read e o seu braço em eval. Com o efeito, a suíte passa com 73/73 em cada linguagem.

APÊNDICE

Sensibilidade à configuração de amostragem



Desvio relativo da razão de tempo: corrida com cota de 5 s e 2.000 amostras (semente 42) contra a mesma semente da campanha, com cota de 20 s. Todos os R^2 da corrida curta ficaram ≥ 0,95.

APÊNDICE

Uso de IA, conforme o Regulamento

Ferramenta: Claude (Anthropic), com a versão informada na declaração do artigo. Finalidades:

- revisão crítica de todas as seções do texto, com sugestões de redação e correção de afirmações factuais;
- redação preliminar dos trechos que reportam e discutem os resultados, revisada pelo autor;
- geração do código-fonte publicado: implementações, suítes de teste, harnesses e scripts de métricas;
- execução da campanha de medição e das métricas de código, em máquina virtual de nuvem.

O conteúdo técnico, os resultados e a redação final são de responsabilidade do autor (Arts. 6º a 9º do Regulamento).